
Clase 173 — JAX y Flax: el stack moderno de Google para DL

Parte: 2 — Deep Learning · Fuente: JAX docs + Flax NNX docs. Duración estimada: 85 min.

· Nivel: Avanzado

Clase 173 — JAX y Flax: el stack moderno de Google para DL

Parte: 2 — Deep Learning · Fuente: JAX docs + Flax NNX docs. Duración estimada: 85 min. · Nivel: Avanzado

Objetivo

Aprender JAX (Google 2018) y Flax (NN library on top of JAX) — el stack que sostiene AlphaFold, Gemini, MaxText, AlphaCode y muchos modelos modernos. Cubrir jit, vmap, pmap, grad, transformaciones funcionales, y Flax NNX (la nueva API 2024, similar a PyTorch).

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Diferenciar JAX (NumPy + autodiff + XLA) de NumPy plano.
- Usar `jax.jit` para compilación XLA (2-10× speedup automático).
- Aplicar `jax.vmap` (vectorización automática) y `jax.grad` (autodiff funcional).
- Construir modelos con Flax NNX (API moderna similar a PyTorch).
- Reconocer cuándo elegir JAX sobre PyTorch (TPU, escala extrema, research).

Temas

- Functional programming: funciones puras, no mutación.
- `jit`, `grad`, `vmap`, `pmap` como transformaciones.
- XLA: compilación a hardware específico (CPU, GPU, TPU).
- PRNG explícito (`jax.random.PRNGKey`).
- Flax: NN library. Antes Linen (functional), ahora NNX (stateful, más parecido a PyTorch).
- Optax: optimizadores.

Definiciones y características

- `jax.numpy`: drop-in NumPy con autodiff + XLA.
- `jit`: compila la función a XLA. Primera vez lento, después rápido.
- `grad`: devuelve función que computa gradiente.
- `vmap`: vectoriza sobre un eje. Como agregar batch dim.
- `pmap`: paraleliza sobre dispositivos (multi-GPU/TPU).
- `PRNGKey`: random determinista. `jax.random.split(key)` para usar varias veces.
- NNX: API stateful de Flax 2024, reemplaza Linen.

Dataset / recursos

- Fashion-MNIST.
- Librerías: `jax`, `jaxlib`, `flax`, `optax`.

Ejercicios

1. JAX basic: `import jax.numpy as jnp; x = jnp.array([1.,2.,3.]); jnp.sum(x**2)`. Comparar contra NumPy.
2. grad: `grad_f = jax.grad(lambda x: x**3); grad_f(2.)` → 12.
3. jit speedup: definir función numérica, medir tiempo con y sin `@jax.jit`. $\geq 5\times$ speedup.
4. vmap: función para una muestra → vmap para procesar batch.
5. Flax NNX MLP: definir modelo, training step, entrenar Fashion-MNIST.

Homework verifiable

Re-entrenar Fashion-MNIST en JAX/Flax:

1. Modelo Flax NNX con 2 capas Dense.
2. Optax adam(1e-3).
3. Training loop con jit.
4. Reportar accuracy + tiempo vs equivalente PyTorch.

Criterio de aceptación: accuracy ≥ 0.87 ; JIT activo (segunda llamada mucho más rápida que primera).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Primera ejecución muy lenta	Compilación XLA. Fix: paciencia, después e
TracerError	Mutación dentro de jit. Fix: pure function
OOM en TPU	Modelo grande sin sharding. Fix: pmap o pj
Random no determinista	Olvidaste split del PRNGKey. Fix: key, sub
Comparar PyTorch vs JAX wall-clock sin JIT	Sin JIT, JAX es lento. Fix: siempre con @j

Preguntas frecuentes

JAX o PyTorch?

PyTorch ecosystem es más grande. JAX brilla en research / TPU / extrema escala. Para 99 % de DL aplicado, PyTorch.

Linen o NNX?

NNX (2024) — más fácil, stateful, parecido a PyTorch. Linen es legacy.

TPU en cloud?

Google Cloud TPU v4/v5e/v5p. Vertex AI lo wrappea. Caro pero performance excelente para batch grande.

Hugging Face soporta JAX?

Sí, muchos modelos tienen versión JAX. Pero la mayoría de la actividad es PyTorch.

¿AlphaFold en JAX?

Sí. JAX brilla en numerical computing (física, chemistry, biology).

Referencias

- JAX docs.
- Flax docs.
- Optax docs.
- Bradbury et al. (2018), JAX: composable transformations of Python+NumPy programs.
- MaxText — LLM training en JAX.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábrilo desde el laboratorio del programa o desde Jupyter.

Siguiente clase

Clase 174 — Entrenamiento a escala con Vertex AI

Apéndice: notebook (primer bloque)

Cubrimos jit, grad, vmap y un MLP con Flax. Fallback en numpy con autograd manual si JAX no está disponible.

```
import numpy as np, time
JAX_OK = False
try:
    import jax, jax.numpy as jnp
    JAX_OK = True
    print(f'JAX {jax.__version__} | devices: {jax.devices()}')
except Exception:
    print('JAX no disponible → fallback con numpy + grad manual')
```

Archivos complementarios

- notebook.ipynb